

Universidad del Valle de Guatemala
CC3069 Computación Paralela y Distribuida, sección 20
Ing. Juan Luis García

Caso 1: Radiografías que no esperan

Caso práctico: CUDA en el mundo real
Informe 1: Propuesta

Grupo 1

Diego López	23747
Nicolás Concuá	23197
Esteban Cárcamo	23016

Guatemala, 4 de noviembre de 2026

1. Problema y usuario

En las unidades de radiología del Instituto Guatemalteco de Seguridad Social (IGSS) se reciben cientos de radiografías de tórax al día y un solo radiólogo de turno las lee en el orden en que llegan. Así, la placa de un afiliado con un neumotórax puede quedar detrás de decenas de estudios normales. Esto afecta al paciente, que espera sin saber que su caso es urgente, al médico de emergencias, que necesita el informe para decidir el tratamiento, y al radiólogo, que no sabe qué placa abrir primero.

La decisión que depende de esto es el orden de lectura. Un neumotórax o un edema pulmonar necesitan atención en minutos, así que una priorización que tarda horas en calcularse ya no sirve. Se propone asignar a cada radiografía un puntaje de urgencia y reordenar la cola cada vez que entra un estudio. El sistema no diagnostica ni sustituye al radiólogo, que sigue leyendo todo, pero empieza por los casos con más probabilidad de ser graves.

2. Conjunto de datos

Se utilizará NIH ChestX-ray14 (Wang et al., 2017), porque son radiografías reales de hospital, sus etiquetas incluyen los hallazgos urgentes del caso y el modelo que se usará fue entrenado con estos datos.

Tabla 1. Datos del conjunto seleccionado.

Fuente	NIH Clinical Center, https://nihcc.app.box.com/v/ChestXray-NIHCC
Tamaño total	112 120 radiografías de 30 805 pacientes en 12 archivos comprimidos (unos 42 GB)
Subconjunto	Archivos images_001 a images_005: unas 45 000 imágenes y 16,5 GB comprimidos, según un espejo público (se confirmará al descargar)
Formato	PNG de 1024 × 1024 píxeles en escala de grises, con etiquetas de 14 hallazgos en un CSV
Licencia	Uso sin restricciones; se debe enlazar el sitio del NIH, citar a Wang et al. (2017) y reconocer al NIH Clinical Center como proveedor

3. Plan de paralelización en GPU

3.1 Tarea a resolver

Las versiones de CPU y de GPU harán el mismo trabajo, con los mismos datos, el mismo modelo y la misma precisión (FP32). Cada radiografía se decodifica, se recorta al centro, se reduce a 224 × 224 píxeles, se normaliza y pasa por la red DenseNet-121 de TorchXRayVision (Cohen et al., 2022; Huang et al., 2017), ya entrenada con radiografías de tórax. El puntaje de urgencia es la mayor probabilidad entre cinco hallazgos críticos: neumotórax, edema, neumonía, derrame y consolidación. Se usa el máximo y no la suma porque el radiólogo prioriza según el hallazgo más grave. Al final, la cola se ordena por puntaje.

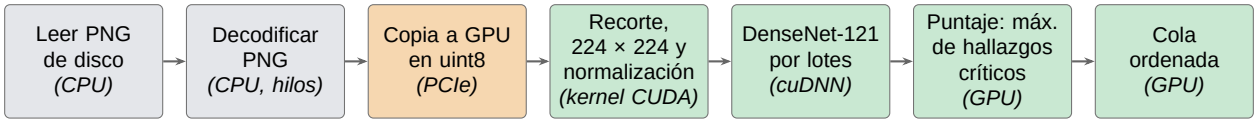


Figura 1. Etapas de la versión GPU: en gris, la CPU; en naranja, la transferencia, y en verde, la GPU.

3.2 Qué se paraleliza

- Las radiografías son independientes entre sí, así que se procesan en lotes de 32 a 64.
- El preprocesado es píxel por píxel: un kernel propio en CuPy (Okuta et al., 2017) usará un hilo por píxel de salida (50 176 por imagen) y se comparará con la misma operación en PyTorch (Paszke et

al., 2019).
- La inferencia son multiplicaciones de matrices muy grandes que cuDNN reparte entre los núcleos de la GPU.
- El puntaje y el ordenamiento de las 45 000 placas también se calculan en la GPU.

La lectura del disco y la decodificación de los PNG se quedan en la CPU en ambas versiones, porque descomprimir cada archivo es secuencial. Esa es la fracción serial que se analizará con la Ley de Amdahl.

3.3 Medición

Ambas versiones correrán en la misma laptop: Intel Core i9-13980HX (24 núcleos y 32 hilos, todos usados por la versión de CPU), 15 GB de RAM y NVIDIA GeForce RTX 4050 Laptop con 6 GB de VRAM. Se reportarán T_{CPU}, T_{GPU}, el speedup T_{CPU}/T_{GPU} y el tiempo de copia a la GPU, medido aparte con eventos de CUDA, como la mediana de cinco ejecuciones tras una de calentamiento. Si la laptop falla, se usará Google Colab y ahí se medirán de nuevo las dos versiones. El código estará en https://github.com/ecarcamo/Second-CUDA-Challenge.

4. Hipótesis y riesgos

Se espera que la inferencia ocupe la mayor parte del tiempo en CPU y que la GPU la acelere al menos diez veces. El speedup total debería ser menor porque la decodificación sigue en la CPU: según la Ley de Amdahl, si esa parte es una fracción s del tiempo, el speedup no puede pasar de $1/s$. Además, enviar cada imagen en uint8 (1 MiB) y no en float32 (4 MiB) debería reducir la copia; se medirán ambas opciones.

Tabla 2. Riesgos identificados y cómo se mitigan.

Riesgo	Mitigación
6 GB de VRAM	Lotes de 32 a 64 imágenes y medición de la memoria pico
15 GB de RAM	Las imágenes decodificadas ocupan unos 45 GB: se leen por lotes
PNG en color o con canal alfa	Conversión a escala de grises en el preprocesado
Modelo entrenado con datos del NIH	Precisión reportada solo sobre la partición oficial de prueba
La Mac de Nicolás no tiene CUDA	Solo se usa para desarrollar; las mediciones se hacen en la laptop

5. Distribución del trabajo

Tabla 3. Responsable de cada etapa.

Integrante	Carné	Responsabilidad
Diego López	23747	Descarga, verificación y preparación del subconjunto y sus etiquetas
Nicolás Concuá	23197	Versión de CPU, desarrollada y probada en su Mac
Esteban Cárcamo	23016	Versión de GPU y mediciones finales en la RTX 4050

Cada integrante coordina una etapa, pero los tres son programadores en todas. El análisis, el informe final y la presentación se harán en conjunto.

Referencias

Cohen, J. P., Viviano, J. D., Bertin, P., Morrison, P., Torabian, P., Guarrera, M., Lungren, M. P., Chaudhari, A., Brooks, R., Hashir, M. y Bertrand, H. (2022). TorchXRayVision: A library of chest X-ray datasets and models. En *Proceedings of the 5th International Conference on Medical Imaging with Deep Learning* (Vol. 172, pp. 231–249). PMLR. https://proceedings.mlr.press/v172/cohen22a.html

Huang, G., Liu, Z., van der Maaten, L. y Weinberger, K. Q. (2017). Densely connected convolutional networks. En *2017 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 2261–2269). https://doi.org/10.1109/CVPR.2017.243

Okuta, R., Unno, Y., Nishino, D., Hido, S. y Loomis, C. (2017). CuPy: A NumPy-compatible library for NVIDIA GPU calculations. En *Workshop on Machine Learning Systems (LearningSys), NIPS 2017*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems 32* (pp. 8024–8035).

Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M. y Summers, R. M. (2017). ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. En *2017 IEEE Conference on Computer Vision and Pattern Recognition*. https://doi.org/10.1109/CVPR.2017.369